



Dipartimento di Elettronica e Informazione

Politecnico di Milano

20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (+39) 02-2399.3400
Fax (+39) 02-2399.3411

High Performance Processors and Systems

May 5, 2020

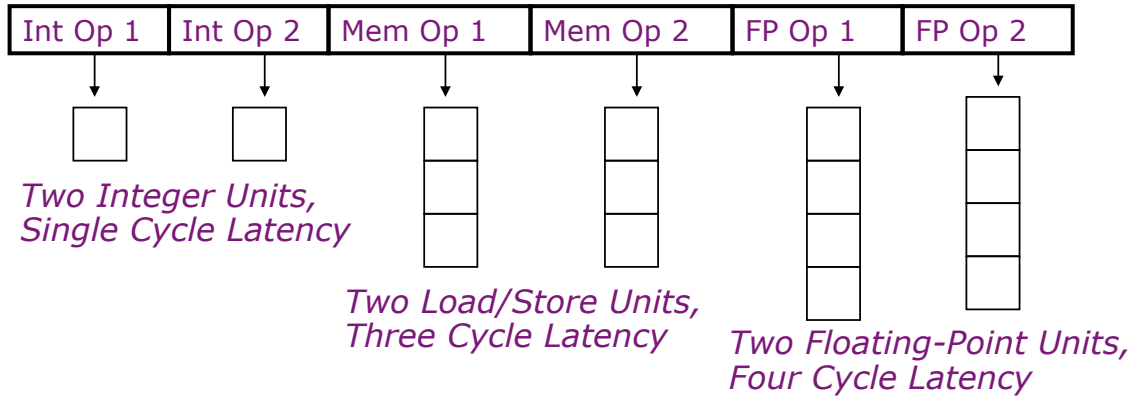
Prof. Marco Santambrogio

Name
Last Name

Problem 1 (25%)	
Problem 2 (25%)	
Question 1 (25%)	
Question 2 (25%)	
Total (100%)	
Extra (3 points)	
Total (x% + 3 points)	

Problem 1

Considering the following VLIW "architecture":



Considering the following portion of assembly, describe the corresponding VLIW code:

```

Loop:    LD F0, 0(R1)
         LD F6, 8(R1)
         LD F10, 16(R1)
         LD F14, 24(R1)
         FADD F4, F0, F2
         FADD F8, F6, F4
         FADD F12, F10, F14
         FADD F16, F14, F2
         SD F4, 0(R1)
         SD F8, 8(R1)
         SD F12, 16(R1)
         ADD R1, 32
         BNE R1, R2, Loop
         SD F16, -8(R1)
    
```

How many FLOPS/cycle?

Answer P1

Int1

Int 2

M1

M2

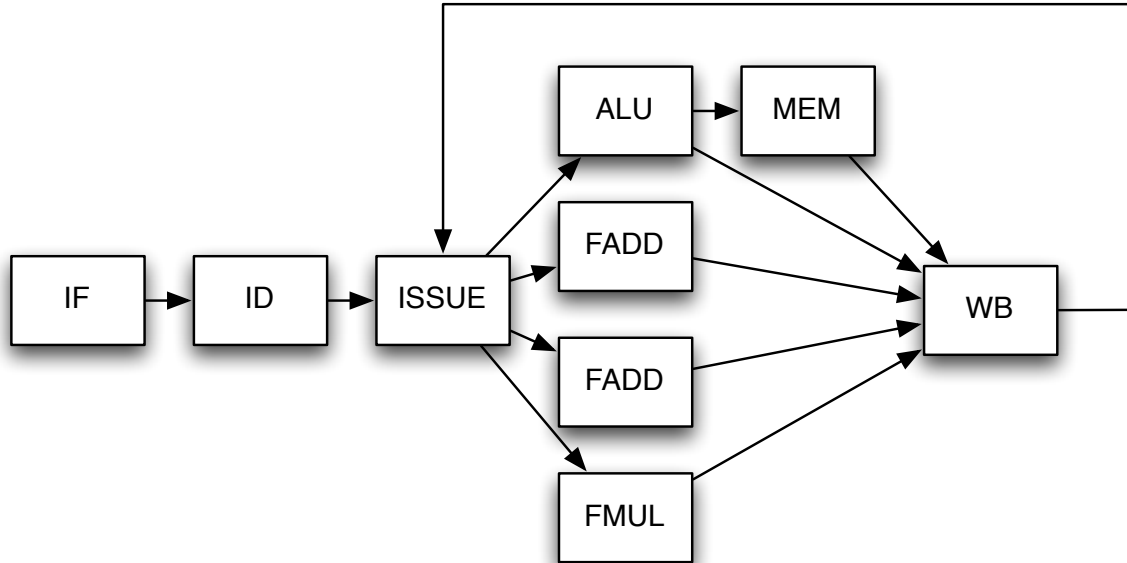
FP+

FPx

[illegible]

Problem 2

In this problem we will examine the execution of a code segment on the following single-issue out-of-order processor:



You can assume that:

- All functional units are pipelined
- ALU operations take 1 cycle
- Memory operations take 5 cycles (includes time in ALU)
- Floating-point add instructions take 3 cycles
- Floating-point multiply instructions take 7 cycles
- There is no register renaming
- Instructions are fetched, decoded and issued in order
- The issue stage is a buffer of unlimited length that holds instructions waiting to start execution
- An instruction will only enter the issue stage if it does not cause a WAR or WAW hazard
- Only one instruction can be issued at a time, and in the case multiple instructions are ready, the oldest one will go first

To simplify book-keeping for this problem we will only track instructions during a few critical stages:

- I - When an instruction enters the issue stage
- E - When an instruction starts execution (enters FU)
- C - When an instruction completes execution (enters WB, results available this cycle)

For this problem we will be using the following code:

I1	L.D	F1, 0(R1)
I2	ADD.D	F1, F1, F2
I3	MUL.D	F2, F3, F3
I4	S.D	F2, 32(R1)
I5	MUL.D	F3, F2, F1
I6	S.D	F3, 0(R1)

Problem 1.A (Note, there is a 1.B in the following page)

Fill in the following table to indicate how the given code will execute. Assume all register values are available at the start of execution.

[illegible][illegible]

Problem 1.B

Why can't an instruction enter the issue stage if it might cause a WAR or WAW hazard even if the earlier instruction causing the hazard has already started execution? (Hint: feel free to use the code from the previous page under adversarial conditions as an example).

Answer

Question 1

Describe (the answer has to be effectively supported) a 1-BHT and a 2-BHT able to execute the following assembly code (R0 is set to 2000, R1 is set to 0)

```
LOOP:      LD      F1  0    R0
           ADDD    F2  F1   F1
           ADDI    R1  R1   100
LOOP2:     MULTD   F2  F2   F1
           SUBI    R1  R1   1
           BNEZ    R1  LOOP2
           SUBI    R0  R0   2
           BNEZ    R0  LOOP
```

The obtained result, in terms of miss predictions, is inline with theoretical characteristics of the two predictors? Please effectively support your answer.

Answer

Question 2

What are possible issues associated with VLIW architecture?

Answer 1: Compilers can detect parallelism only in local portions of the code

Answer 2: Larger code size

Answer 3: Increased hw complexity

Answer 4: Less code portability

For each of the answers motivate why it is TRUE or FALSE

EXTRA

Provide a motivation for each answer TRUE or FALSE.

1. *Global code scheduling is more effective than loop unrolling in control-flow intensive code*
2. *In software pipelining each iteration executes instructions chosen from different iterations of the original loop*
3. *Software pipelining replicates less code than loop unrolling*
4. *Software pipelining needs more registers than loop unrolling*
5. *Trace scheduling obtains the best performance when there is an imbalance in the frequency of the traces*
6. *In Trace scheduling traces are scheduled without taking branches into consideration*

